

PhishTrainer

React Project Scaffold

Frontend Systems — Prof. Dr. Peter Braun — SS 2026

Project Structure

The project was set up with Vite, which was chosen over Create React App for its faster startup and build times. Components are kept separate from views: views represent full screens connected to routes, while components are smaller, reusable building blocks shared across multiple views. This separation keeps the codebase understandable as more functionality is added in later milestones.

```
src/  
  components/  Navbar, StatCard, EmailCard, ResultBanner, HistoryItem  
  views/      Dashboard, Scenario, Feedback, History  
  App.jsx     routing setup  
db.json      mock data for json-server
```

Key Components and Views

All four views are implemented as routed pages using React Router. Navigation between Dashboard and History is available via the Navbar at all times, while Scenario and Feedback are reached through the training flow itself (Start training → Submit decision → Next scenario). This mirrors the screen flow defined in the design draft.

Four reusable components were extracted so far: StatCard displays a single statistic on the Dashboard, EmailCard renders the simulated phishing email, ResultBanner shows whether a verdict was correct, and HistoryItem represents a single past session entry. Currently, all displayed data (XP values, email content, history entries) is hardcoded directly in the views to verify that the layout and component composition work as intended. No state management or API calls have been wired up yet at this stage.

Technical Decisions

- **Vite** for fast startup and build times during development.
- **React Router** for client-side routing between the four main views without full page reloads.
- **json-server** serves the mock scenario data (`db.json`) as a REST API on a separate local port. This allows the application to perform real asynchronous HTTP requests in later milestones without building a custom backend.

Open Issues and Next Steps

The current scaffold is intentionally minimal but fully functional in terms of setup and navigation. The following work remains for upcoming milestones:

- Connect the views to the json-server API instead of using hardcoded data.
- Add state management (e.g. `useState/useEffect`) to track the user's progress, score, and verdicts.
- Implement the suspicious-element marking interaction on the Scenario screen.
- Add CSS styling based on the wireframes from the design draft document.

AI Tools

This document was drafted with the support of Claude (Anthropic) for wording and structure. All technical decisions and code were reviewed by the team.